

EPAM SDET Interview

JSON → POJO Practice + Complete Answers

JSON → POJO

10 exercises with complete Java POJO answers

The answer section includes no-argument constructors, parameterised constructors where useful, fields, and representative getters/setters. For longer examples, getters/setters are marked for completion to keep the workbook readable.

Key mappings

JSON object → Java POJO

JSON array → List<T>

Nested object → another POJO

Array of objects → List<CustomPOJO>

JSON → POJO = deserialization

POJO → JSON = serialization

1. Simple JSON → POJO

Complete answer • Beginner

Java POJO Answer

```
public class User {  
    private int id;  
    private String name;  
    private String email;  
    private int age;  
  
    public User() {}  
  
    public User(int id, String name, String email, int age) {  
        this.id = id;  
        this.name = name;  
        this.email = email;  
        this.age = age;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
  
    public String getEmail() { return email; }  
    public void setEmail(String email) { this.email = email; }  
  
    public int getAge() { return age; }  
    public void setAge(int age) { this.age = age; }  
}
```

2. Nested JSON → POJO

Complete answer • Beginner

Java POJO Answer

```
public class User {
    private int id;
    private String name;
    private Address address;

    public User() {}

    public User(int id, String name, Address address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public Address getAddress() { return address; }
    public void setAddress(Address address) { this.address = address; }
}

public class Address {
    private String city;
    private String state;
    private String country;

    public Address() {}

    public Address(String city, String state, String country) {
        this.city = city;
        this.state = state;
        this.country = country;
    }

    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }

    public String getState() { return state; }
    public void setState(String state) { this.state = state; }

    public String getCountry() { return country; }
    public void setCountry(String country) { this.country = country; }
}
```

3. JSON Array → List<String>

Complete answer • Intermediate

Java POJO Answer

```
import java.util.List;

public class User {
    private int id;
    private String name;
    private List<String> skills;

    public User() {}

    public User(int id, String name, List<String> skills) {
        this.id = id;
        this.name = name;
        this.skills = skills;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public List<String> getSkills() { return skills; }
    public void setSkills(List<String> skills) { this.skills = skills; }
}
```

4. Array of Objects → List<Employee>

Complete answer • Intermediate

Java POJO Answer

```
import java.util.List;

public class Department {
    private String department;
    private List<Employee> employees;

    public Department() {}

    public Department(String department, List<Employee> employees) {
        this.department = department;
        this.employees = employees;
    }

    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }

    public List<Employee> getEmployees() { return employees; }
    public void setEmployees(List<Employee> employees) { this.employees = employees; }
}

public class Employee {
    private int id;
    private String name;
    private int salary;

    public Employee() {}

    public Employee(int id, String name, int salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }
}
```

5. Nested Object + Array

Complete answer • Intermediate

Java POJO Answer

```
import java.util.List;

public class Order {
    private int id;
    private String orderName;
    private Customer customer;
    private List<OrderItem> items;

    public Order() {}

    public Order(int id, String orderName, Customer customer,
        List<OrderItem> items) {
        this.id = id;
        this.orderName = orderName;
        this.customer = customer;
        this.items = items;
    }

    // getters and setters for all fields
}

public class Customer {
    private int id;
    private String name;
    private String email;

    public Customer() {}

    public Customer(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    // getters and setters
}

public class OrderItem {
    private int productId;
    private String productName;
    private int quantity;

    public OrderItem() {}

    public OrderItem(int productId, String productName, int quantity) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
    }

    // getters and setters
}
```

6. Deeply Nested JSON

Complete answer • Intermediate

Java POJO Answer

```
public class User {
    private int id;
    private String name;
    private Address address;

    public User() {}

    // getters and setters
}

public class Address {
    private String city;
    private Location location;

    public Address() {}

    // getters and setters
}

public class Location {
    private double latitude;
    private double longitude;

    public Location() {}

    // getters and setters
}
```

7. Boolean + Null + Decimal

Complete answer • Intermediate

Java POJO Answer

```
import java.math.BigDecimal;

public class User {
    private int id;
    private String name;
    private boolean active;
    private BigDecimal salary;
    private String phone;

    public User() {}

    public User(int id, String name, boolean active,
                BigDecimal salary, String phone) {
        this.id = id;
        this.name = name;
        this.active = active;
        this.salary = salary;
        this.phone = phone;
    }

    // getters and setters
}
```

8. JSON Names Different from Java Names

Complete answer • Advanced

Java POJO Answer

```
import com.fasterxml.jackson.annotation.JsonProperty;

public class User {

    @JsonProperty("user_id")
    private int userId;

    @JsonProperty("first_name")
    private String firstName;

    @JsonProperty("last_name")
    private String lastName;

    @JsonProperty("email_address")
    private String emailAddress;

    public User() {}

    // getters and setters
}
```

9. Realistic API Response

Complete answer • Advanced

Java POJO Answer

```
import java.math.BigDecimal;
import java.util.List;

public class OrderResponse {
    private String status;
    private String message;
    private OrderData data;

    public OrderResponse() {}

    // getters and setters
}

class OrderData {
    private int orderId;
    private Customer customer;
    private List<Product> products;

    public OrderData() {}

    // getters and setters
}

class Customer {
    private int id;
    private String name;

    public Customer() {}

    // getters and setters
}

class Product {
    private int id;
    private String name;
    private BigDecimal price;
    private int quantity;

    public Product() {}

    // getters and setters
}
```

10. EPAM-Level Challenge

Complete answer • Advanced

Java POJO Answer

```
import java.util.List;

public class ApiResponse {
    private boolean success;
    private Data data;

    public ApiResponse() {}

    // getters and setters
}

class Data {
    private int userId;
    private String userName;
    private List<String> roles;
    private Profile profile;
    private List<Project> projects;

    public Data() {}

    // getters and setters
}

class Profile {
    private String email;
    private String phone;
    private Address address;

    public Profile() {}

    // getters and setters
}

class Address {
    private String city;
    private String state;

    public Address() {}

    // getters and setters
}

class Project {
    private int projectId;
    private String projectName;
    private boolean active;

    public Project() {}

    // getters and setters
}
```

EPAM POJO Interview Cheat Sheet

What you should be able to identify immediately

Simple field	String / int / boolean / BigDecimal
Nested object	Create another POJO and use it as the field type
Array of strings	List<String>
Array of objects	List<CustomPOJO>
JSON name mismatch	@JsonProperty
Request body	POJO → JSON serialization
API response	JSON → POJO deserialization
No-arg constructor	Useful/commonly expected by Jackson for deserialization
Parameterised constructor	Convenient for creating request objects

Interview sequence

Given JSON → identify objects → identify arrays → choose Java types → create POJO classes → constructors → getters/setters → Jackson/Rest Assured.